

CSS Animation Property

Complete Guide: Easy English Explanation

Contents

1	Introduction	2
1.1	What is CSS Animation?	2
1.2	Why Use CSS Animation?	2
1.3	Animation vs Transition	2
2	Core Animation Concepts	4
2.1	How Animations Work	4
2.1.1	Step 1: Define Keyframes	4
2.1.2	Step 2: Apply Animation to Element	4
2.1.3	Step 3: Animation Plays	4
2.2	Animation Timing: Keyframes	4
2.2.1	Keyframe Syntax	4
2.3	Main Animation Properties	5
2.3.1	1. animation-name	5
2.3.2	2. animation-duration	5
2.3.3	3. animation-timing-function	5
2.3.4	4. animation-delay	6
2.3.5	5. animation-iteration-count	6
2.3.6	6. animation-direction	6
2.3.7	7. animation-fill-mode	7
2.3.8	8. animation-play-state	7
3	The Animation Shorthand	8
3.1	Using animation Shorthand	8
3.2	Shorthand Syntax	8
4	Practical Examples	9
4.1	Example 1: Simple Fade In	9
4.2	Example 2: Sliding Animation	9
4.3	Example 3: Rotating Loader	10
4.4	Example 4: Pulse Effect	10
4.5	Example 5: Color Change Animation	10
4.6	Example 6: Bounce Animation	12
4.7	Example 7: Staggered Animation (Multiple Elements)	12
4.8	Example 8: Combined Transform Animation	12
4.9	Example 9: Text Animation	13
4.10	Example 10: Hover-Triggered Animation	14

5	Animation Timing Functions	15
5.1	Built-in Timing Functions	15
5.1.1	linear	15
5.1.2	ease (default)	15
5.1.3	ease-in	15
5.1.4	ease-out	15
5.1.5	ease-in-out	15
5.2	Custom cubic-bezier()	15
5.3	steps() Function	16
6	Common Mistakes	17
6.1	Mistake 1: Forgetting @keyframes	17
6.2	Mistake 2: Using animation-fill-mode Incorrectly	17
6.3	Mistake 3: Animation Not Playing	18
6.4	Mistake 4: Performance Issues	19
7	Browser Support	20
7.1	Compatibility	20
7.2	Vendor Prefixes	20
8	Advanced Topics	21
8.1	Controlling Animations with JavaScript	21
8.2	Multiple Animations	21
8.3	Performance Optimization	22
9	Quick Reference	23
9.1	Animation Properties Summary	23
9.2	Common Keyframe Percentages	23
9.3	Complete Animation Setup Checklist	23
9.4	Best Practices	24
9.5	Accessibility Consideration	24
9.6	Common Use Cases	24

Chapter 1

Introduction

1.1 What is CSS Animation?

CSS Animation is a way to make elements move, change color, rotate, scale, or transform smoothly over a period of time without using JavaScript. It brings your web pages to life with motion and visual effects.

Think of animation like a flip book:

- Each page shows a slightly different position
- When you flip through quickly, it looks like smooth motion
- CSS animations work the same way - showing different states smoothly

1.2 Why Use CSS Animation?

CSS Animation provides:

- Smooth, performant motion (GPU-accelerated)
- Engaging user experiences
- Loading indicators and spinners
- Hover effects and interactions
- Auto-playing animations (without user action)
- Better performance than JavaScript animations
- Easier code than complex JavaScript

1.3 Animation vs Transition

Feature	Animation	Transition
Trigger	Auto-play or on hover	Requires state change
Duration	Configurable	Configurable
Repeats	Can loop infinitely	Only once
Control	More keyframes	Only 2 states (from/to)

Simple comparison:

- **Transition:** Smooth change between two states (A to B)
- **Animation:** Smooth motion through multiple states (A to B to C to D)

Chapter 2

Core Animation Concepts

2.1 How Animations Work

2.1.1 Step 1: Define Keyframes

Keyframes are like checkpoints that define how an element looks at different times:

```
@keyframes slide {
  0% {
    transform: translateX(0);
  }
  50% {
    transform: translateX(100px);
  }
  100% {
    transform: translateX(200px);
  }
}
```

2.1.2 Step 2: Apply Animation to Element

Then you tell an element to use those keyframes:

```
.box {
  animation: slide 2s;
  /* Play "slide" animation over 2 seconds */
}
```

2.1.3 Step 3: Animation Plays

The animation smoothly transitions between each keyframe checkpoint.

2.2 Animation Timing: Keyframes

2.2.1 Keyframe Syntax

```
@keyframes animation-name {
  0% {
    /* Starting state */
    transform: translateX(0);
  }
  50% {
    /* Middle state */
    transform: translateX(100px);
  }
  100% {
    /* Ending state */
    transform: translateX(200px);
  }
}
```

You can use any percentage from 0

```
@keyframes fade {
  from {
    opacity: 0;
  }
  to {
    opacity: 1;
  }
}
```

2.3 Main Animation Properties

2.3.1 1. animation-name

Specifies which @keyframes to use:

```
.element {
  animation-name: slide;
  /* Uses @keyframes slide {...} */
}
```

2.3.2 2. animation-duration

How long the animation takes (in seconds or milliseconds):

```
.element {
  animation-duration: 2s;      /* 2 seconds */
  /* OR */
  animation-duration: 2000ms; /* 2000 milliseconds */
}
```

2.3.3 3. animation-timing-function

How the animation progresses (speed curve):

```
.element {
  animation-timing-function: ease;
  /* Other options: linear, ease-in, ease-out,
    ease-in-out, cubic-bezier() */
}
```

Common values:

- **linear** - Constant speed throughout
- **ease** - Slow start, fast middle, slow end (default)
- **ease-in** - Slow start, fast end
- **ease-out** - Fast start, slow end
- **ease-in-out** - Slow start and end, fast middle

2.3.4 4. animation-delay

Delay before animation starts:

```
.element {
  animation-delay: 1s;
  /* Wait 1 second, then play animation */
}
```

2.3.5 5. animation-iteration-count

How many times the animation repeats:

```
.element {
  animation-iteration-count: 1;          /* Play once */
  /* OR */
  animation-iteration-count: 3;          /* Play 3 times */
  /* OR */
  animation-iteration-count: infinite; /* Loop forever */
}
```

2.3.6 6. animation-direction

Direction the animation plays:

```
.element {
  animation-direction: normal;
  /* Other options: reverse, alternate,
    alternate-reverse */
}
```

Options:

- **normal** - Play forward each time
- **reverse** - Play backwards

- **alternate** - Forward, then backward, then forward
- **alternate-reverse** - Backward, then forward, then backward

2.3.7 7. animation-fill-mode

What happens before and after animation:

```
.element {  
  animation-fill-mode: forwards;  
  /* Other options: none, backwards, both */  
}
```

Options:

- **none** - No styles before/after (default)
- **forwards** - Keep final keyframe styles after animation
- **backwards** - Apply first keyframe before animation starts
- **both** - Apply both (before and after)

2.3.8 8. animation-play-state

Play or pause the animation:

```
.element {  
  animation-play-state: running;  
  /* Or: paused */  
}  
  
.element:hover {  
  animation-play-state: paused;  
  /* Pause on hover */  
}
```

Chapter 3

The Animation Shorthand

3.1 Using animation Shorthand

Instead of writing each property separately, use the **animation** shorthand:

```
/* Individual properties */
.element {
  animation-name: slide;
  animation-duration: 2s;
  animation-timing-function: ease;
  animation-delay: 1s;
  animation-iteration-count: infinite;
  animation-direction: alternate;
}

/* Shorthand (same result) */
.element {
  animation: slide 2s ease 1s infinite alternate;
}
```

3.2 Shorthand Syntax

```
animation: name duration timing-function delay
           iteration-count direction fill-mode;
```

Order matters:

1. animation-name (required)
2. animation-duration (required)
3. animation-timing-function (optional, default: ease)
4. animation-delay (optional, default: 0s)
5. animation-iteration-count (optional, default: 1)
6. animation-direction (optional, default: normal)
7. animation-fill-mode (optional, default: none)

Chapter 4

Practical Examples

4.1 Example 1: Simple Fade In

```
@keyframes fadeIn {
  0% {
    opacity: 0;
  }
  100% {
    opacity: 1;
  }
}

.box {
  animation: fadeIn 1s ease-in;
  /* Fade from invisible to visible over 1 second */
}
```

4.2 Example 2: Sliding Animation

```
@keyframes slideIn {
  0% {
    transform: translateX(-100px);
    opacity: 0;
  }
  100% {
    transform: translateX(0);
    opacity: 1;
  }
}

.element {
  animation: slideIn 0.5s ease-out forwards;
  /* Slide from left, keep final position */
}
```

4.3 Example 3: Rotating Loader

```
@keyframes spin {
  0% {
    transform: rotate(0deg);
  }
  100% {
    transform: rotate(360deg);
  }
}

.loader {
  width: 50px;
  height: 50px;
  border: 4px solid #f3f3f3;
  border-top: 4px solid #3498db;
  border-radius: 50%;
  animation: spin 1s linear infinite;
  /* Spin continuously, constant speed */
}
```

4.4 Example 4: Pulse Effect

```
@keyframes pulse {
  0% {
    transform: scale(1);
  }
  50% {
    transform: scale(1.1);
  }
  100% {
    transform: scale(1);
  }
}

.button {
  animation: pulse 1s ease-in-out infinite;
  /* Grows and shrinks continuously */
}
```

4.5 Example 5: Color Change Animation

```
@keyframes colorShift {
  0% {
    background: #ff6b6b;
  }
  25% {
    background: #4ecdc4;
  }
}
```

```
50% {  
    background: #45b7d1;  
}  
75% {  
    background: #f7b731;  
}  
100% {  
    background: #ff6b6b;  
}  
}  
  
.box {  
    width: 200px;  
    height: 200px;  
    animation: colorShift 4s ease-in-out infinite;  
    /* Cycle through colors */  
}
```

4.6 Example 6: Bounce Animation

```
@keyframes bounce {
  0%, 100% {
    transform: translateY(0);
  }
  50% {
    transform: translateY(-30px);
  }
}

.ball {
  width: 30px;
  height: 30px;
  background: #3498db;
  border-radius: 50%;
  animation: bounce 0.6s ease-in-out infinite;
  /* Ball bounces up and down */
}
```

4.7 Example 7: Staggered Animation (Multiple Elements)

```
@keyframes slideUp {
  0% {
    opacity: 0;
    transform: translateY(30px);
  }
  100% {
    opacity: 1;
    transform: translateY(0);
  }
}

.item {
  animation: slideUp 0.5s ease-out forwards;
}

/* Stagger each item with different delay */
.item:nth-child(1) { animation-delay: 0s; }
.item:nth-child(2) { animation-delay: 0.1s; }
.item:nth-child(3) { animation-delay: 0.2s; }
.item:nth-child(4) { animation-delay: 0.3s; }
```

4.8 Example 8: Combined Transform Animation

```
@keyframes morphBox {
  0% {
    width: 100px;
    height: 100px;
```

```
    background: #3498db;
    border-radius: 0;
}
25% {
    width: 120px;
    height: 80px;
}
50% {
    width: 80px;
    height: 120px;
    background: #e74c3c;
    border-radius: 50%;
}
75% {
    width: 100px;
    height: 100px;
}
100% {
    width: 100px;
    height: 100px;
    background: #3498db;
    border-radius: 0;
}
}

.shape {
    animation: morphBox 3s ease-in-out infinite;
    /* Shape changes form and color */
}
```

4.9 Example 9: Text Animation

```
@keyframes blinkText {
    0%, 49% {
        opacity: 1;
    }
    50%, 100% {
        opacity: 0;
    }
}

.blink {
    animation: blinkText 1s ease-in-out infinite;
    /* Text blinks on and off */
}

/* Typing effect */
@keyframes typing {
    0% {
        width: 0;
    }
    100% {
```

```
    width: 100%;
  }
}

.typewriter {
  animation: typing 2s steps(20, end) 1;
  /* Types out character by character */
  overflow: hidden;
  white-space: nowrap;
}
```

4.10 Example 10: Hover-Triggered Animation

```
@keyframes wiggle {
  0%, 100% {
    transform: rotate(0deg);
  }
  25% {
    transform: rotate(-5deg);
  }
  75% {
    transform: rotate(5deg);
  }
}

.icon {
  transition: transform 0.3s;
}

.icon:hover {
  animation: wiggle 0.5s ease-in-out;
  /* Shake/wiggle on hover */
}
```


Chapter 5

Animation Timing Functions

5.1 Built-in Timing Functions

5.1.1 linear

Constant speed from start to finish.

```
animation: slide 2s linear;
```

5.1.2 ease (default)

Slow start, fast middle, slow end.

```
animation: slide 2s ease;
```

5.1.3 ease-in

Slow start, accelerates to the end.

```
animation: slide 2s ease-in;
```

5.1.4 ease-out

Starts fast, decelerates to the end.

```
animation: slide 2s ease-out;
```

5.1.5 ease-in-out

Slow start and end, fast in middle.

```
animation: slide 2s ease-in-out;
```

5.2 Custom cubic-bezier()

For precise control, use cubic-bezier:

```
animation: slide 2s cubic-bezier(0.25, 0.1, 0.25, 1);
```

5.3 steps() Function

Create frame-by-frame animations:

```
/* Animate in 10 discrete steps */  
animation: type 2s steps(10, end);
```

Chapter 6

Common Mistakes

6.1 Mistake 1: Forgetting @keyframes

Wrong:

```
.element {  
  animation: slide 2s;  
  /* "slide" keyframes don't exist! */  
}
```

Correct:

```
@keyframes slide {  
  0% { transform: translateX(0); }  
  100% { transform: translateX(100px); }  
}  
  
.element {  
  animation: slide 2s;  
}
```

6.2 Mistake 2: Using animation-fill-mode Incorrectly

Wrong:

```
.element {  
  animation: fade 1s;  
  /* Element returns to original state after! */  
}
```

Correct:

```
.element {  
  animation: fade 1s forwards;  
  /* Keeps final state after animation */  
}
```

6.3 Mistake 3: Animation Not Playing

Common reasons:

- Missing `@keyframes` definition
- Animation name doesn't match
- Duration too short to notice
- Check browser console for errors

Debug checklist:

`@keyframes` defined?

Names match (case-sensitive)?

Duration long enough?

Element visible on page?

Browser supports CSS animations?

6.4 Mistake 4: Performance Issues

Bad (causes jank):

```
@keyframes badAnimation {  
  0% { width: 0; height: 0; }  
  100% { width: 500px; height: 500px; }  
}
```

Better (GPU-accelerated):

```
@keyframes goodAnimation {  
  0% { transform: scale(0); }  
  100% { transform: scale(1); }  
}
```

Animate these for best performance:

- transform
- opacity
- filter

Avoid animating:

- width, height
- top, left, bottom, right
- margin, padding

Chapter 7

Browser Support

7.1 Compatibility

CSS Animations are supported in all modern browsers:

- Chrome 26+
- Firefox 16+
- Safari 9+
- Edge (all versions)
- Opera 15+
- iOS Safari 9+
- Chrome Mobile 26+

7.2 Vendor Prefixes

For older browser support:

```
@-webkit-keyframes slide {
  0% { transform: translateX(0); }
  100% { transform: translateX(100px); }
}

@keyframes slide {
  0% { transform: translateX(0); }
  100% { transform: translateX(100px); }
}

.element {
  -webkit-animation: slide 2s;
  animation: slide 2s;
}
```

Chapter 8

Advanced Topics

8.1 Controlling Animations with JavaScript

```
.element {  
  animation-play-state: paused;  
}  
  
.element.playing {  
  animation-play-state: running;  
}
```

JavaScript:

```
const element = document.querySelector('.element');  
  
// Play animation  
element.classList.add('playing');  
  
// Pause animation  
element.classList.remove('playing');  
  
// Restart animation  
element.style.animation = 'none';  
setTimeout(() => {  
  element.style.animation = 'slide 2s';  
}, 10);
```

8.2 Multiple Animations

```
.element {  
  animation: slide 2s ease-in,  
            fade 1s ease-out,  
            rotate 3s linear infinite;  
  /* Multiple animations play simultaneously */  
}
```

8.3 Performance Optimization

```
.animated-element {  
  will-change: transform;  
  animation: slide 2s ease-out;  
  /* Tell browser to optimize for this */  
}  
  
/* Clean up after animation */  
@keyframes slideCleanup {  
  100% {  
    will-change: auto;  
  }  
}
```


Chapter 9

Quick Reference

9.1 Animation Properties Summary

Property	Purpose
animation-name	Which keyframes to use
animation-duration	How long animation takes
animation-timing-function	Speed curve
animation-delay	Delay before starting
animation-iteration-count	How many times to play
animation-direction	Forward, reverse, or alternate
animation-fill-mode	Before/after animation styles
animation-play-state	Play or pause
animation	Shorthand for all above

9.2 Common Keyframe Percentages

- 0% or from - Starting state
- 50% - Middle/halfway point
- 100% or to - Ending state

9.3 Complete Animation Setup Checklist

Define @keyframes

Name the animation

Set starting state (0%)

Set ending state (100%)

Add animation to element

Set duration (`animation-duration`)

Choose timing function (`ease`, `linear`, etc.)

- Set iteration count (**infinite** for loops)
- Set fill-mode (**forwards** to keep final state)
- Test in multiple browsers

9.4 Best Practices

1. Use **transform** and **opacity** for best performance
2. Set appropriate delays for staggered effects
3. Use **ease-in-out** for natural motion
4. Add **animation-fill-mode: forwards** to keep final state
5. Test animations on different devices
6. Use **will-change** for performance-critical animations
7. Keep animations under 3 seconds for UI feedback
8. Provide **prefers-reduced-motion** option for accessibility

9.5 Accessibility Consideration

Respect user motion preferences:

```
@media (prefers-reduced-motion: reduce) {  
  .animated-element {  
    animation: none;  
  }  
}
```

9.6 Common Use Cases

- Loading spinners
- Page transitions
- Hover effects
- Auto-playing hero animations
- Icon animations
- Button feedback
- Scroll animations
- Notification alerts
- Skeleton screens
- Infinite loops (backgrounds, banners)